



Problem A [5 points]: Backpropagation and Weight Initialization Part 1

Fit the neural network at [this link](#) for about 250 iterations, and then do the same for the neural network at [this link](#). Both networks have the same architecture and use ReLU activations. The only difference between the two is how the layer weights were initialized – you can examine the layer weights by hovering over the edges between neurons.

Give a mathematical justification, based on what you know about the backpropagation algorithm and the ReLU function, for the difference in the performance of the two networks.

Solution A.:

Let's check one hidden unit

$$z = w^T x + b, \quad a = \text{ReLU}(z) = \max(0, z)$$

with Back Prop.

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot x \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial \text{ReLU}}{\partial z}$$

For ReLU:

$$\text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \quad \left. \begin{array}{l} \text{For } z=0 \text{ its not} \\ \text{DEFINED,} \\ \text{most implementations} \\ \text{TAKE } z=0 \end{array} \right\}$$

Therefore if all weights initialized to exactly zero, then

$$\text{If } w=0 \text{ and } b=0 \text{ then } z=0 \rightarrow a = \text{ReLU}(z) = 0$$

$$\text{ReLU}'(z)=0 \rightarrow \frac{\partial \mathcal{L}}{\partial w} = 0, \text{ so we are stuck in zero}$$

forever. So we don't see any updates in weights.

The second example, when we take weights to then make some units' start $z > 0 \rightarrow \text{ReLU}'(z) = 1$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a} \cdot x \neq 0$$

Problem B [5 points]: Backpropagation and Weight Initialization Part 2

Reset the two demos from part i (there is a reset button to the left of the "Run" button), change the activation functions of the neurons to sigmoid instead of ReLU, and train each of them for 4000 iterations.

Explain the differences in the models learned, and the speed at which they were learned, from those of part i in terms of the backpropagation algorithm and the sigmoid function.

We can observe that in the case where we have weights $\neq 0$ the convergence now is slower compared to when we used ReLU. On the other hand, for the second case where we use weights $= 0$, near epoch 3600 the NN started updating some of the weights, and the loss decreased from 0.5 to 0.412 (Test loss).

Why is slower when we use sigmoid?

Back prop is a product of many derivatives:

$$z^{(l)} = w^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma(z^{(l)})$$

$$f^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}} = (w^{(l+1)})^T f^{(l+1)} \underbrace{\sigma'(z^{(l)})}$$

Activation $\rightarrow$ For Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$, $\sigma'(z) = \sigma(z)(1-\sigma(z))$

So $\sigma \in (0,1) \rightarrow \sigma'(z)$ is max when $\sigma(z) = \frac{1}{2}$

$$\text{Therefore } \sigma'(z) \leq \frac{1}{2} \rightarrow \begin{cases} z=0 \rightarrow \sigma'(0) = 0.25 \\ z=2 \rightarrow \sigma'(2) \approx 0.33 \\ z=4 \rightarrow \sigma'(4) \approx 0.018 \end{cases}$$

Therefore if these activations are small then, GRADIENTS SHOWS LATER BY LAYER. ReLU keeps GRADIENTS "ALIVE", we ALWAYS MULTIPLY GRADIENTS BY 1 (IF $z > 0$).

Problem C: [10 Points]

When training any model using SGD, it's important to shuffle your data to avoid correlated samples. To illustrate one reason for this that is particularly important for ReLU networks, consider a dataset of 1000 points, 500 of which have positive (+1) labels, and 500 of which have negative (-1) labels. What happens if we train a fully-connected network with ReLU activations using SGD, looping through all the negative examples before any of the positive examples? (Hint: this is called the "dying ReLU" problem.)

Solution C:

TRAINING with SGD on all negative values first biases early updates to identify the negative class. In the ReLU network, these updates can push many hidden-pre activations " ≤ 0 " below " 0 " $\rightarrow \text{ReLU}(z) = 0 \rightarrow \text{ReLU}'(z) = 0$. This can deactivate units and prevent later positive examples from effectively updating earlier layers. Then learning positive cases could be slower or may not fully recover. That is why shuffling avoids this by keeping SGD GRADIENTS REPRESENTATIVE of both classes.

Problem D: Approximating Functions Part 1 [7 Points]

Draw or describe a fully-connected network with ReLU units that implements the OR function on two 0/1-valued inputs, x_1 and x_2 . Your networks should contain the minimum number of hidden units possible. The OR function $\text{OR}(x_1, x_2)$ is defined as:

$$\text{OR}(1, 0) \geq 1$$

$$\text{OR}(0, 1) \geq 1$$

$$\text{OR}(1, 1) \geq 1$$

$$\text{OR}(0, 0) = 0$$

Your network need only produce the correct output when $x_1 \in \{0, 1\}$ and $x_2 \in \{0, 1\}$ (as described in the examples above).

Solution D.:

I can use the text

$$\hat{y} = \text{ReLU}(\underbrace{4x_1 + 4x_2 - 2}_z)$$

$$w_1 = 4, \quad w_2 = 4, \quad b = -2$$

Then.

$$(0, 0) \rightarrow z = -2 \rightarrow \text{ReLU}(z) = 0$$

$$(0, 1) \rightarrow z = 2 \rightarrow \text{ReLU}(z) = 1$$

$$(1, 0) \rightarrow z = 2 \rightarrow \text{ReLU}(z) = 1$$

$$(1, 1) \rightarrow z = 6 \rightarrow \text{ReLU}(z) = 1$$

} This is a
valid OR
Network

Problem E: Approximating Functions Part 2 [8 Points]

What is the minimum number of fully-connected layers (with ReLU units) needed to implement an XOR of two 0/1-valued inputs x_1, x_2 ? Recall that the XOR function is defined as:

$$\text{XOR}(1, 0) \geq 1$$

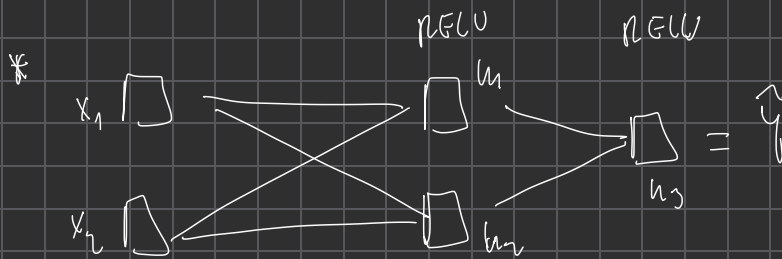
$$\text{XOR}(0, 1) \geq 1$$

$$\text{XOR}(0, 0) = \text{XOR}(1, 1) = 0$$

For the purposes of this problem, we say that a network f computes the XOR function if $f(x_1, x_2) = \text{XOR}(x_1, x_2)$ when $x_1 \in \{0, 1\}$ and $x_2 \in \{0, 1\}$ (as described in the examples above).

Explain why a network with fewer layers than the number you specified cannot compute XOR.

Solution E.:



$$z_1 = x_1 - x_2 \rightarrow h_1(z_1) = \text{ReLU}(x_1 - x_2)$$

$$z_2 = x_2 - x_1 \rightarrow h_2(z_2) = \text{ReLU}(x_2 - x_1)$$

$$h_3 = \text{ReLU}(h_1 + h_2) = \uparrow$$

Let's check:

$$(1, 0) \rightarrow \begin{cases} z_1 = 1 \rightarrow h_1(1) = 1 \\ z_2 = -1 \rightarrow h_2(-1) = 0 \end{cases} \quad \left\{ \begin{array}{l} h_3 = \text{ReLU}(1+0) = 1 \end{array} \right.$$

$$(0, 1) \rightarrow \begin{cases} z_1 = -1 \rightarrow h_1(-1) = 0 \\ z_2 = 1 \rightarrow h_2(1) = 1 \end{cases} \quad \left\{ \begin{array}{l} h_3 = \text{ReLU}(0+1) = 1 \end{array} \right.$$

For $(0, 0)$, $(1, 1)$ is the same
 $\left. \begin{array}{l} z_1 = 0 \rightarrow h_1(0) = 0 \\ z_2 = 0 \rightarrow h_2(0) = 0 \end{array} \right\} \quad \left\{ \begin{array}{l} h_3 = \text{ReLU}(0+0) = 0 \end{array} \right.$
 Therefore we only need 2 fully connected layers, we need to combine the activations in layer 1, otherwise we can not get XOR. With fewer layers we can only get "OR" or "AND"

1 My code

My solutions and code are in [the next link](#)

2 Deep Learning Principles [35 Points]

Relevant materials: lectures on deep learning

For problems A and B, we'll be utilizing the [Tensorflow Playground](#) to visualize/fit a neural network.

Problem A [5 points]: Backpropagation and Weight Initialization Part 1

Fit the neural network at [this link](#) for about 250 iterations, and then do the same for the neural network at [this link](#). Both networks have the same architecture and use ReLU activations. The only difference between the two is how the layer weights were initialized – you can examine the layer weights by hovering over the edges between neurons.

Give a mathematical justification, based on what you know about the backpropagation algorithm and the ReLU function, for the difference in the performance of the two networks.

Solution A.:

Problem B [5 points]: Backpropagation and Weight Initialization Part 2

Reset the two demos from part i (there is a reset button to the left of the “Run” button), change the activation functions of the neurons to sigmoid instead of ReLU, and train each of them for 4000 iterations.

Explain the differences in the models learned, and the speed at which they were learned, from those of part i in terms of the backpropagation algorithm and the sigmoid function.

Solution B.:

Problem C: [10 Points]

When training any model using SGD, it's important to shuffle your data to avoid correlated samples. To illustrate one reason for this that is particularly important for ReLU networks, consider a dataset of 1000 points, 500 of which have positive (+1) labels, and 500 of which have negative (-1) labels. What happens if we train a fully-connected network with ReLU activations using SGD, looping through all the negative examples before any of the positive examples? (Hint: this is called the "dying ReLU" problem.)

Solution C:

Problem D: Approximating Functions Part 1 [7 Points]

Draw or describe a fully-connected network with ReLU units that implements the OR function on two 0/1-valued inputs, x_1 and x_2 . Your networks should contain the minimum number of hidden units possible. The OR function $\text{OR}(x_1, x_2)$ is defined as:

$$\text{OR}(1, 0) \geq 1$$

$$\text{OR}(0, 1) \geq 1$$

$$\text{OR}(1, 1) \geq 1$$

$$\text{OR}(0, 0) = 0$$

Your network need only produce the correct output when $x_1 \in \{0, 1\}$ and $x_2 \in \{0, 1\}$ (as described in the examples above).

Solution D.:

Problem E: Approximating Functions Part 2 [8 Points]

What is the minimum number of fully-connected layers (with ReLU units) needed to implement an XOR of two 0/1-valued inputs x_1, x_2 ? Recall that the XOR function is defined as:

$$\text{XOR}(1, 0) \geq 1$$

$$\text{XOR}(0, 1) \geq 1$$

$$\text{XOR}(0, 0) = \text{XOR}(1, 1) = 0$$

For the purposes of this problem, we say that a network f computes the XOR function if $f(x_1, x_2) = \text{XOR}(x_1, x_2)$ when $x_1 \in \{0, 1\}$ and $x_2 \in \{0, 1\}$ (as described in the examples above).

Explain why a network with fewer layers than the number you specified cannot compute XOR.

Solution E.:

3 Depth vs Width on the MNIST Dataset [25 Points]

MNIST is a classic dataset in computer vision. It consists of images of handwritten digits (0 - 9) and the correct digit classification. In this problem you will implement a deep network using PyTorch to classify MNIST digits. Specifically, you will explore what it really means for a network to be “deep”, and how depth vs. width impacts the classification accuracy of a model. You will be allowed at most N hidden units, and will be expected to design and implement a deep network that meets some performance baseline on the MNIST dataset.

Problem A: Installation [2 Points]

Before any modeling can begin, PyTorch must be installed. PyTorch is an automatic differentiation framework that is widely used in machine learning research. We will also need the **torchvision** package, which will make downloading the MNIST dataset much easier.

To install both packages, follow the steps on

<https://pytorch.org/get-started/locally/#start-locally>. Select the ‘Stable’ build and your system information. We highly recommend using Python 3.6+. CUDA is not required for this class, but it is necessary if you want to do GPU-accelerated deep learning in the future.

Once you have finished installing, write down the version numbers for both **torch** and **torchvision** that you have installed.

Solution A:

torch: 2.7.1+cu118

torchvision: 0.22.1+cu118

Problem B: The Data [3 Points]

Load the MNIST dataset using torchvision; see the problem 2 sample code for how.

Image inputs in PyTorch are generally 3D tensors with the shape (no. of channels, height, width). Examine the input data. What are the height and width of the images? What do the values in each array index represent? How many images are in the training set? How many are in the testing set? You can use the `imshow` function in matplotlib if you'd like to see the actual pictures (see the sample code).

Solution B.: *The MNIST images have the following properties:*

- **Height:** 28 pixels
- **Width:** 28 pixels
- **Shape:** (1, 28, 28) representing (channels, height, width)

*The values in each array index represent **grayscale pixel intensities**, ranging from 0.0 (black) to 1.0 (white). The `transforms.ToTensor()` transformation normalizes the original 0–255 pixel values to the 0.0–1.0 range.*

- **Training set:** 60,000 images
- **Test set:** 10,000 images

mama

Problem C: Modeling Part 1 [8 Points]

Using PyTorch's "Sequential" model class, build a deep network to classify the handwritten digits. You may **only** use the following layers:

- **Linear:** A fully-connected layer
- **ReLU (activation):** Sets negative inputs to 0
- **Softmax (activation):** Rescales input so that it can be interpreted as a (discrete) probability distribution.
- **Dropout:** Takes some probability and at every iteration sets weights to zero at random with that probability (effectively regularization)

A sample network with 20 hidden units is in the sample code file. (Note: activations, Dropout, and your last Linear layer do not count toward your hidden unit count, because the final layer is "observed" and not *hidden*.)

Use categorical cross entropy as your loss function. There are also a number of optimizers you can use (an optimizer is just a fancier version of SGD), and feel free to play around with them, but RMSprop and Adam are the most popular and will probably work best. You also should find the batch size and number of epochs that give you the best results (default is batch size = 32, epochs=10).

Look at the sample code to see how to train your model. PyTorch should make it very easy to tinker with your network architecture.

Your task. Using at most 100 hidden units, build a network using only the allowed layers that achieves test accuracy of at least 0.975. Turn in the code of your model as well as the best test accuracy that it achieved.

Hint: for best results on this problem and the two following problems, normalize the input vectors by dividing the values by 255 (as the pixel values range from 0 to 255).

Solution C:

Model Architecture: 2 hidden layers with $64 + 36 = 100$ hidden units total.

Final Test Accuracy: 0.9753 (97.53%)

```
1 # Deeper network with lower dropout for better accuracy
2 model = nn.Sequential(
3     nn.Flatten(),
4     nn.Linear(784, 64),      # First hidden layer: 64 units
5     nn.ReLU(),
6     nn.Dropout(0.2),
7     nn.Linear(64, 36),      # Second hidden layer: 36 units (total: 64+36=100)
8     nn.ReLU(),
```

```
9     nn.Dropout(0.2),
10     nn.Linear(36, 10)      # Output layer (not counted as hidden)
11 )
12 model = model.to(device)
13 print(model)
14 print(f"Total hidden units: 64 + 36 = 100")
15
16 # loss function
17 loss_fn = nn.CrossEntropyLoss()
18 # optimizer
19 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
20 # number of epochs to train
21 num_epochs = 20
22
23 for epoch in range(num_epochs):
24     model.train()
25     for batch_idx, (data, target) in enumerate(train_loader):
26         data, target = data.to(device), target.to(device)
27         # forward pass
28         output = model(data)
29         loss = loss_fn(output, target)
30         # backward pass and optimization
31         optimizer.zero_grad()
32         loss.backward()
33         optimizer.step()
34
35     # evaluate on test set
36     model.eval()
37     correct = 0
38     total = 0
39     with torch.no_grad():
40         for data, target in test_loader:
41             data, target = data.to(device), target.to(device)
42             output = model(data)
43             _, predicted = torch.max(output.data, 1)
44             total += target.size(0)
45             correct += (predicted == target).sum().item()
46     test_accuracy = correct / total
47     print(f'Epoch {epoch+1}/{num_epochs}, Test Accuracy: {test_accuracy:.4f}')
48 # Final test accuracy
49 print(f'Final Test Accuracy: {test_accuracy:.4f}')
```

Problem D: Modeling Part 2 [6 Points]

Repeat problem C, except that now you may use 200 hidden units and must build a model with at least 2 hidden layers that achieves test accuracy of at least 0.98.

Solution D: Final Test Accuracy: 0.9807 (98.07%)

```
1 # Deeper network with lower dropout for better accuracy
2 model = nn.Sequential(
3     nn.Flatten(),
4     nn.Linear(784, 100),    # Hidden layer 1: 100 units
5     nn.ReLU(),
6     nn.Dropout(0.1),
7     nn.Linear(100, 60),    # Hidden layer 2: 60 units
8     nn.ReLU(),
9     nn.Dropout(0.1),
10    nn.Linear(60, 40),      # Hidden layer 3: 40 units (total: 100+60+40=200)
11    nn.ReLU(),
12    nn.Dropout(0.1),
13    nn.Linear(40, 10)      # Output layer
14 )
15
16 model = model.to(device) # Add this line
17 print(model)
18 print(f"Total hidden units: 100 + 60 + 40 = 200")
19
20 # loss function
21 loss_fn = nn.CrossEntropyLoss()
22 # optimizer
23 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
24 # number of epochs to train
25 num_epochs = 20
26
27 for epoch in range(num_epochs):
28     model.train()
29     for batch_idx, (data, target) in enumerate(train_loader):
30         data, target = data.to(device), target.to(device) # Add this line
31         # data = data / 255.0
32         # forward pass
33         output = model(data)
34         loss = loss_fn(output, target)
35         # backward pass and optimization
36         optimizer.zero_grad()
37         loss.backward()
38         optimizer.step()
39
40     # evaluate on test set
41     model.eval()
42     correct = 0
43     total = 0
```

```
44     with torch.no_grad():
45         for data, target in test_loader:
46             data, target = data.to(device), target.to(device) # Add this line
47             # data = data / 255.0
48             output = model(data)
49             _, predicted = torch.max(output.data, 1)
50             total += target.size(0)
51             correct += (predicted == target).sum().item()
52     test_accuracy = correct / total
53     print(f'Epoch {epoch+1}/{num_epochs}, Test Accuracy: {test_accuracy:.4f}')
54 # Final test accuracy
55 print(f'Final Test Accuracy: {test_accuracy:.4f}')
```

Problem E: Modeling Part 3 [6 Points]

Repeat problem C, except that now you may use 1000 hidden units and must build a model with at least 3 hidden layers that achieves test accuracy of at least 0.983.

Solution E: Final Test Accuracy: 0.9861 (98.61%)

```
1
2 # 1000 hidden units across 4 hidden layers: 400+300+200+100
3 model = nn.Sequential(
4     nn.Flatten(),
5
6     nn.Linear(784, 400),          # Hidden layer 1: 400 units
7     nn.BatchNorm1d(400),
8     nn.ReLU(),
9     nn.Dropout(0.15),
10
11    nn.Linear(400, 300),          # Hidden layer 2: 300 units
12    nn.BatchNorm1d(300),
13    nn.ReLU(),
14    nn.Dropout(0.15),
15
16    nn.Linear(300, 200),          # Hidden layer 3: 200 units
17    nn.BatchNorm1d(200),
18    nn.ReLU(),
19    nn.Dropout(0.15),
20
21    nn.Linear(200, 100),          # Hidden layer 4: 100 units
22    nn.BatchNorm1d(100),
23    nn.ReLU(),
24    nn.Dropout(0.1),
25
26    nn.Linear(100, 10)            # Output layer
27 )
28 model = model.to(device)
29 print(model)
30 print(f"Total hidden units: 400 + 300 + 200 + 100 = 1000")
31
32
33 # loss function
34 loss_fn = nn.CrossEntropyLoss()
35 # optimizer
36 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
37 # number of epochs to train
38 num_epochs = 20
39
40 for epoch in range(num_epochs):
41     model.train()
42     for batch_idx, (data, target) in enumerate(train_loader):
43         data, target = data.to(device), target.to(device) # Add this line
```

```
44     # data = data / 255.0
45     # forward pass
46     output = model(data)
47     loss = loss_fn(output, target)
48     # backward pass and optimization
49     optimizer.zero_grad()
50     loss.backward()
51     optimizer.step()
52
53     # evaluate on test set
54     model.eval()
55     correct = 0
56     total = 0
57     with torch.no_grad():
58         for data, target in test_loader:
59             data, target = data.to(device), target.to(device) # Add this line
60             # data = data / 255.0
61             output = model(data)
62             _, predicted = torch.max(output.data, 1)
63             total += target.size(0)
64             correct += (predicted == target).sum().item()
65     test_accuracy = correct / total
66     print(f'Epoch {epoch+1}/{num_epochs}, Test Accuracy: {test_accuracy:.4f}')
67 # Final test accuracy
68 print(f'Final Test Accuracy: {test_accuracy:.4f}')
```

4 Convolutional Neural Networks [40 Points]

Problem A: Zero Padding [5 Points]

Consider a convolutional network in which we perform a convolution over each 8×8 patch of a 20×20 input image. It is common to zero-pad input images to allow for convolutions past the edges of the images. An example of zero-padding is shown below:

0	0	0	0	0
0	5	4	9	0
0	7	8	7	0
0	10	2	1	0
0	0	0	0	0

Figure: A convolution being applied to a 2×2 patch (the red square) of a 3×3 image that has been zero-padded to allow convolutions past the edges of the image.

What is one benefit and one drawback to this zero-padding scheme (in contrast to an approach in which we only perform convolutions over patches entirely contained within an image)?

Solution A:

No Padding : The output shrinks to $(20-8+1) \times (20-8+1) = 13 \times 13$ With Padding : The output remains 20×20

Benefit: With zero-padding we preserve the spatial dimensions of the output feature map. Witouth padding, the output shings from 20×20 to 13×13 , losing 7 pixels on each axis per layer. Over multiple convolution layers, this compounds and the spatial resolution collapses quickly.

Drawback: With zero-padding we introduce artificial values, zeros, that do not correspond to real image content. Convolutions computer near the vorders are therefores operating over a mixture of real pixel data and fabricated zeros, which can produce lower quality or misleading features at the edges. Therefore, the network may learn to detect patterns that are partly artifacts of the padding rather than meaningful image features.

5 x 5 Convolutions

Consider a single convolutional layer, where your input is a 32×32 pixel, RGB image. In other words, the input is a $32 \times 32 \times 3$ tensor. Your convolution has:

- Size: $5 \times 5 \times 3$
- Filters: 8
- Stride: 1
- No zero-padding

Problem B [2 points]: What is the number of parameters (weights) in this layer, including a bias term?

Solution B.:

We have $5 \times 5 \times 3 = 75$ weights per filter, 1 bias term per filter, and 8 filters. Total parameters = $8 \times (75 + 1) = 608$

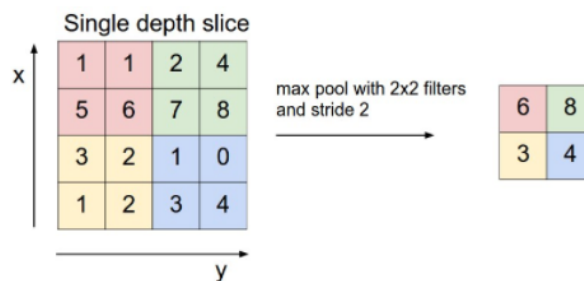
Problem C [3 points]: What is the shape of the output tensor?

Solution C.: *we need to calculate the output height and width using the formula: $\text{Output Size} = (\text{Input Size} - \text{Filter Size}) / \text{Stride} + 1$ $\text{Output Height} = (32 - 5) / 1 + 1 = 28$ $\text{Output Width} = (32 - 5) / 1 + 1 = 28$ $\text{Output Depth} = \text{Number of Filters} = 8$ So the output tensor shape is (28, 28, 8)*

Max/Average Pooling

Pooling is a downsampling technique for reducing the dimensionality of a layer's output. Pooling iterates across patches of an image similarly to a convolution, but pooling and convolutional layers compute their outputs differently: given a pooling layer B with preceding layer A , the output of B is some function (such as the max or average functions) applied to patches of A 's output.

Below is an example of max-pooling on a 2-D input space with a 2×2 filter (the max function is applied to 2×2 patches of the input) and a stride of 2 (so that the sampled patches do not overlap):



Average pooling is similar except that you would take the average of each patch as its output instead of the maximum.

Consider the following 4 matrices:

$$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

Problem D [3 points]:

Apply 2×2 average pooling with a stride of 2 to each of the above images.

Solution D.:

Matrix 1:

$$\begin{bmatrix} 1 & 0.5 \\ 0.5 & 0.25 \end{bmatrix}$$

$$\begin{bmatrix} 0.5 & 1 \\ 0.25 & 0.5 \end{bmatrix}$$

$$\begin{bmatrix} 0.25 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$
$$\begin{bmatrix} 0.5 & 0.25 \\ 1 & 0.5 \end{bmatrix}$$

Problem E [3 points]:

Apply 2×2 max pooling with a stride of 2 to each of the above images.

Solution E.:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Problem F [4 points]:

Consider a scenario in which we wish to classify a dataset of images of various animals, taken at various angles/locations and containing small amounts of noise (e.g. some pixels may be missing). Why might pooling be advantageous given these distortions in our dataset?

Solution F: *In this case, pooling is advantageous because it provides spatial invariance and robustness to noise.*

If we apply 2x2 average pooling, we summarize a local region into a single value, this means that if an animal shifts slightly in the frame, lets say a cat appears a few pixels to the left in one image versus another, the pooled output remain largely unchanged. This is translation invariance. For noise robustness, if some pixels are missing or corrupted, the max operation simply selects the strongest activation in the region, ignoring the noise. Additionally, pooling progressively reduces the spatial dimensions of the data, which forces the network to focus on higher-level features (shapes, textures, body parts) rather than exact pixel positions. This abstraction helps the model generalize better to new images of animals taken from different angles.

PyTorch implementation

Problem G [20 points]:

Using PyTorch “Sequential” model class as you did in 2C, build a deep *convolutional* network to classify the handwritten digits in MNIST. You are now allowed to use the following layers (but **only** the following):

- **Linear:** A fully-connected layer
 - In convolutional networks, Linear (also called dense) layers are typically used to knit together higher-level feature representations.
 - Particularly useful to map the 2D features resulting from the last convolutional layer to categories for classification (like the 1000 categories of ImageNet or the 10 categories of MNIST).
 - Inefficient use of parameters and often overkill: for A input activations and B output activations, number of parameters needed scales as $O(AB)$.
- **Conv2d:** A 2-dimensional convolutional layer
 - The bread and butter of convolutional networks, conv layers impose a translational-invariance prior on a fully-connected network. By sliding filters across the image to form another image, conv layers perform “coarse-graining” of the image.
 - Networking several convolutional layers in succession helps the convolutional network knit together more abstract representations of the input. As you go higher in a convolutional network, activations represent pixels, then edges, colors, and finally objects.
 - More efficient use of parameters. For N filters of $K \times K$ size on an input of size $L \times L$, the number of parameters needed scales as $O(NK^2)$. When N, K are small, this can often beat the $O(L^4)$ scaling of a Linear layer applied to the L^2 pixels in the image.
- **MaxPool2d:** A 2-dimensional max-pooling layer
 - Another way of performing “coarse-graining” of images, max-pool layers are another way of ignoring finer-grained details by only considering maximum activations over small patches of the input.
 - Drastically reduces the input size. Useful for reducing the number of parameters in your model.
 - Typically used immediately following a series of convolutional-activation layers.
- **BatchNorm2d:** Performs batch normalization (Ioffe and Szegedy, 2014). Normalizes the activations of previous layer to standard normal (mean 0, standard deviation 1).
 - Accelerates convergence and improves performance of model, especially when saturating non-linearities (sigmoid) are used.
 - Makes model less sensitive to higher learning rates and initialization, and also acts as a form of regularization.

- Typically used immediately before nonlinearity (Activation) layers.
- **Dropout:** Takes some probability and at every iteration sets weights to zero at random with that probability
 - An effective form of regularization. During training, randomly selecting activations to shut off forces network to build in redundancies in the feature representation, so it does not rely on any single activation to perform classification.
- **ReLU (activation):** Sets negative inputs to 0
- **Softmax (activation):** Rescales input so that it can be interpreted as a (discrete) probability distribution.
- **Flatten:** Flattens any tensor into a single vector (required in order to pass a 2D tensor output from a convolutional layer as input into Linear layers)

Your tasks. Build a network with only the allowed layers that achieves **test accuracy of at least 0.985**. You are required to use categorical cross entropy as your loss function and to train for 10 epochs with a batch size of 32. Note: your model must have fewer than 1 million parameters, as measured by the method given in the sample code. Everything else can change: optimizer (RMSProp, Adam, ???), initial learning rates, dropout probabilities, layerwise regularizer strengths, etc. You are not required to use all of the layers, but *you must have at least one dropout layer and one batch normalization layer in your final model*. Try to figure out the best possible architecture and hyperparameters given these building blocks!

In order to design your model, you should train your model for 1 epoch (batch size 32) and look at the final **test accuracy** after training. This should take no more than 10 minutes, and should give you an immediate sense for how fast your network converges and how good it is.

Set the probabilities of your dropout layers to 10 equally-spaced values $p \in [0, 1]$, train for 1 epoch, and report the final model accuracies for each.

You can perform all of your hyperparameter validation in this way: vary your parameters and train for an epoch. After you're satisfied with the model design, you should train your model for the full 10 epochs.

In your submission. Turn in the code of your model, the test accuracy for the 10 dropout probabilities $p \in [0, 1]$, and the final test accuracy when your model is trained for 10 epochs. We should have everything needed to reproduce your results.

Discuss what you found to be the most effective strategies in designing a convolutional network. Which regularization method was most effective (dropout, layerwise regularization, batch norm)?

Do you foresee any problem with this way of validating our hyperparameters? If so, why?

Hints:

- You are provided with a sample network that achieves a high accuracy. Starting with this network, modify some of the regularization parameters (layerwise regularization strength, dropout probabilit-

ities) to see if you can maximize the test accuracy. You can also add layers or modify layers (e.g. changing the convolutional kernel sizes, number of filters, stride, dilation, etc.) so long as the total number of parameters remains under the cap of 1 million.

- You may want to read up on successful convolutional architectures, and emulate some of their design principles. Please cite any idea you use that is not your own.
- To better understand the function of each layer, check the PyTorch documentation.
- Linear layers take in single vector inputs (ex: $(784,)$) but Conv2D layers take in tensor inputs (ex: $(28, 28, 1)$): width, height, and channels. Using the transformation `transforms.ToTensor()` when loading the dataset will reshape the training/test X to a 4-dimensional tensor (ex: $(num_examples, width, height, channels)$) and normalize values. For the MNIST dataset, $channels=1$. Typical color images have 3 color channels, 1 for each color in RGB.
- If your model is running slowly on your CPU, try making each layer smaller and stacking more layers so you can leverage deeper representations.
- Other useful CNN design principles:
 - CNNs perform well with many stacked convolutional layers, which develop increasingly large-scale representations of the input image.
 - Dropout ensures that the learned representations are robust to some amount of noise.
 - Batch norm is done after a convolutional or dense layer and immediately prior to an activation/nonlinearity layer.
 - Max-pooling is typically done after a series of convolutions, in order to gradually reduce the size of the representation.
 - Finally, the learned representation is passed into a dense layer (or two), and then filtered down to the final softmax layer.

Solution G.: *The most effective strategy was incorporating batch normalization after every convolutional layer, which stabilized and accelerated training. BatchNorm stabilized training dramatically, allowing the model to reach 98.7% validation accuracy after just a single epoch.*

I increased the model capacity. Going from 8 filters per layer to 32 and 64 filters, and using two convolutional layers per block instead of one, gave the network enough representational power to achieve high accuracy. The model remained under 1 million parameters limited at approximately 470k parameters,

Dropout played a minor supporting role — the sweep showed that low dropout values ($p = 0.0$ to 0.4) all performed similarly well, while high values ($p \geq 0.889$) produced bad results, collapsing accuracy to random chance (~ 10 – 13%).

Do I foresee any problem with this way of validating our hyperparameters? The main problem is related to the dropout selection process itself. When we trained 10 models with different dropout probabilities for a single

epoch each, and choosing the one that performed best on the test set, we leaked information from the test set into our model selection process. This means the reported test accuracy is an optimistically biased estimate of true generalization performance, because our design decisions were influenced by test set results. In this case it would have been better k -fold cross-validation on the training set for hyperparameter tuning, and reserve the test set exclusively for a single final evaluation.

Table 1: Test accuracy after 1 epoch for different dropout probabilities

Dropout p	Test Accuracy
0.000	0.9918
0.111	0.9909
0.222	0.9878
0.333	0.9902
0.444	0.9894
0.556	0.9874
0.667	0.9811
0.778	0.9750
0.889	0.1243
1.000	0.1328

```
1  import torch.nn as nn
2
3  class MyModel(nn.Module):
4      def __init__(self, dropout_p=0.3):
5          super().__init__()
6          self.net = nn.Sequential(
7              # Block 1
8              nn.Conv2d(1, 32, 3, padding=1),
9              nn.BatchNorm2d(32),
10             nn.ReLU(),
11             nn.Conv2d(32, 32, 3, padding=1),
12             nn.BatchNorm2d(32),
13             nn.ReLU(),
14             nn.MaxPool2d(2),
15             nn.Dropout(dropout_p),
16
17             # Block 2
18             nn.Conv2d(32, 64, 3, padding=1),
19             nn.BatchNorm2d(64),
20             nn.ReLU(),
21             nn.Conv2d(64, 64, 3, padding=1),
22             nn.BatchNorm2d(64),
23             nn.ReLU(),
24             nn.MaxPool2d(2),
25             nn.Dropout(dropout_p),
26
27             # Classifier
28             nn.Flatten(),
29             nn.Linear(64 * 7 * 7, 128),
30             nn.BatchNorm1d(128),
31             nn.ReLU(),
32             nn.Dropout(dropout_p),
33             nn.Linear(128, 10)
34         )
35
36     def forward(self, x):
37         return self.net(x)
```

```
1  import numpy as np
2
3  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
4  print(f"Using device: {device}")
5
6  dropout_results = {}
7
8  for p in np.linspace(0, 1, 10):
9      model = MyModel(dropout_p=p).to(device)
10     optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
11     criterion = nn.CrossEntropyLoss()
12
13     # train 1 epoch
14     model.train()
```

```
15     for i, data in enumerate(training_data_loader):
16         images, labels = data
17         images, labels = images.to(device), labels.to(device)
18         optimizer.zero_grad()
19         output = model(images)
20         loss = criterion(output, labels)
21         loss.backward()
22         optimizer.step()
23
24     # evaluate
25     model.eval()
26     test_correct, test_total = 0, 0
27     with torch.no_grad():
28         for data in test_data_loader:
29             images, labels = data
30             images, labels = images.to(device), labels.to(device)
31             output = model(images)
32             _, predicted = torch.max(output.data, 1)
33             test_total += labels.size(0)
34             test_correct += (predicted == labels).sum().item()
35
36     acc = test_correct / test_total
37     dropout_results[p] = acc
38     print(f"p={p:.3f} --> Test Acc: {acc:.4f}")
39
40 print("\n--- Results ---")
41 for p, acc in dropout_results.items():
42     print(f"p={p:.3f}: {acc:.4f}")
```

```
1 # Train the model for 10 epochs, iterating on the data in batches
2 n_epochs = 10
3
4 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 model = MyModel(dropout_p=0.111).to(device)
6 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
7 criterion = nn.CrossEntropyLoss()
8
9 # store metrics
10 training_accuracy_history = np.zeros([n_epochs, 1])
11 training_loss_history = np.zeros([n_epochs, 1])
12 validation_accuracy_history = np.zeros([n_epochs, 1])
13 validation_loss_history = np.zeros([n_epochs, 1])
14
15 for epoch in range(n_epochs):
16     print(f'Epoch {epoch+1}/10:', end='')
17     train_total = 0
18     train_correct = 0
19     # train
20     model.train()
21     for i, data in enumerate(training_data_loader):
22         images, labels = data
23         images, labels = images.to(device), labels.to(device)
```

```
24     optimizer.zero_grad()
25     # forward pass
26     output = model(images)
27     # calculate categorical cross entropy loss
28     loss = criterion(output, labels)
29     # backward pass
30     loss.backward()
31     optimizer.step()
32
33     # track training accuracy
34     _, predicted = torch.max(output.data, 1)
35     train_total += labels.size(0)
36     train_correct += (predicted == labels).sum().item()
37     # track training loss
38     training_loss_history[epoch] += loss.item()
39     # progress update after 180 batches (~1/10 epoch for batch size 32)
40     if i % 180 == 0: print('.',end='')
41     training_loss_history[epoch] /= len(training_data_loader)
42     training_accuracy_history[epoch] = train_correct / train_total
43     print(f'\n\tloss: {training_loss_history[epoch]:0.4f}, acc: {training_accuracy_history[
44         epoch]:0.4f}',end='')
45
46     # validate
47     test_total = 0
48     test_correct = 0
49     with torch.no_grad():
50         model.eval()
51         for i, data in enumerate(test_data_loader):
52             images, labels = data
53             images, labels = images.to(device), labels.to(device) # <-- added
54             # forward pass
55             output = model(images)
56             # find accuracy
57             _, predicted = torch.max(output.data, 1)
58             test_total += labels.size(0)
59             test_correct += (predicted == labels).sum().item()
60             # find loss
61             loss = criterion(output, labels)
62             validation_loss_history[epoch] += loss.item()
63             validation_loss_history[epoch] /= len(test_data_loader)
64             validation_accuracy_history[epoch] = test_correct / test_total
65     print(f', val loss: {validation_loss_history[epoch]:0.4f}, val acc: {
66         validation_accuracy_history[epoch]:0.4f}')
```